

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2026/2027

**MODUL 15- Component, Navigation, dan Styling di React Native +
Proyek Mobile**

Nama Mahasiswa : Aisyah Ayu Salsabila
NIM : 23051430001
Kelas : A2_Teknik Industri
Tanggal Praktikum : 29 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 15 (Pertemuan 15)
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Aisyah Ayu Salsabila
NIM	23051430001
Kelas	A2_Teknik Industri

B. TUJUAN PEMBELAJARAN

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi Live Server		Perubahan web realtime
4	Laptop	Windows 11	Prasarana Praktikum
5	Modul Aplikasi Web dan Mobile	2025/2026	Manual Praktikum
6	Console browser dan ekstensi Quokka.js		JavaScript alat pemroses data

7.	Node.js		Untuk npm
8.	Expo.Go		Aplikasi di HP
9.	Project Expo		Pertemuan 14

D. DASAR TEORI

1. Navigasi dalam Aplikasi Mobile Berbeda dengan Web

Dalam pengembangan web, perpindahan halaman biasanya menggunakan URL (misalnya /about, /product/1). Namun, dalam aplikasi mobile, tidak ada konsep URL bar. Sebagai gantinya, mobile menggunakan mekanisme Stack Navigation (Tumpukan Layar). Setiap layar baru akan "ditumpuk" di atas layar sebelumnya, seperti tumpukan piring. Saat pengguna menekan tombol kembali, layar tersebut akan dikeluarkan dari tumpukan (*pop*). React Navigation adalah library standar untuk mengimplementasikan pola ini di React Native.

2. React Navigation dan Native Stack

React Navigation adalah library navigasi yang fleksibel dan berbasis JavaScript. Untuk navigasi tumpukan, kita menggunakan `@react-navigation/native-stack` yang memanfaatkan komponen native (`react-native-screens`) untuk performa tinggi. Berbeda dengan navigasi berbasis URL, navigasi ini menggunakan fungsi seperti `navigate('NamaLayar')` dan mengirim data melalui `params`.

3. Passing Data Antar Layar

Di aplikasi mobile, data sering dikirim dari daftar (list) ke halaman detail. Ketika pengguna menekan sebuah item, data seperti id, nama, atau harga dapat dikirim sebagai parameter (`params`) ke layar berikutnya. Layar penerima (`DetailScreen`) kemudian mengambil data tersebut menggunakan `route.params`. Ini mirip dengan mengirim data melalui state di React web, tetapi khusus untuk navigasi.

4. Komponen Pendukung Navigasi

Komponen	Fungsi dalam Navigasi
<code>NavigationContainer</code>	Komponen induk yang mengelola state navigasi seluruh aplikasi.
<code>createNativeStackNavigator</code>	Fungsi untuk membuat tumpukan navigator.
<code>Stack.Screen</code>	Mendefinisikan setiap layar (misal: Home, Detail) beserta opsinya.
<code>navigation.navigate()</code>	Fungsi untuk berpindah ke layar lain.
<code>route.params</code>	Objek yang berisi data yang dikirim dari layar sebelumnya.

D.1 Konsep Utama

1. Navigasi Stack di React Native menggunakan mekanisme tumpukan layar, berbeda dengan web yang menggunakan URL. Layar baru ditumpuk di atas layar lama, dan tombol kembali melakukan pop pada stack.
2. react-navigation/native-stack adalah library untuk mengimplementasikan navigasi, dilengkapi dengan fungsi `navigation.navigate()` untuk berpindah layar dan `route.params` untuk mengirim data antar layar.
3. Passing Data Antar Layar memungkinkan data item (seperti nama, harga, stok) dikirim dari halaman daftar (HomeScreen) ke halaman detail (DetailScreen) melalui parameter.
4. Struktur Folder memisahkan setiap layar ke file terpisah dalam folder `screens/` (misal: `HomeScreen.js` dan `DetailScreen.js`) agar kode lebih terstruktur.

D.2 Keterkaitan dengan Teknik Industri

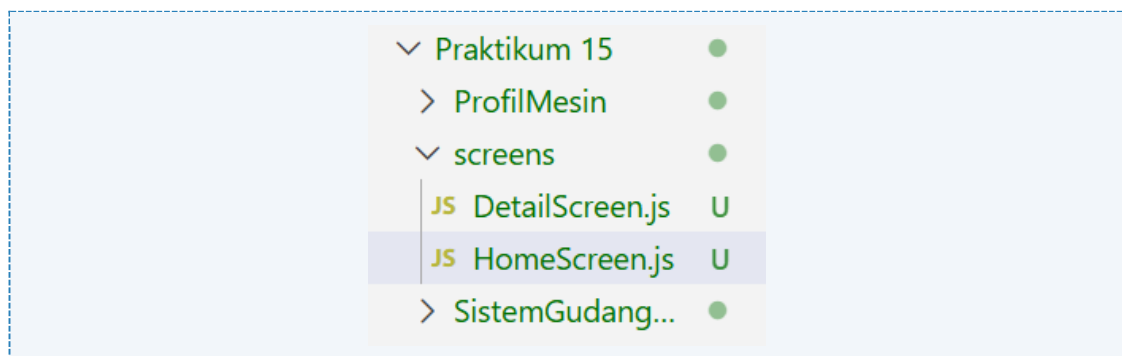
Aplikasi inventori dengan navigasi stack memungkinkan operator pabrik atau gudang mengecek stok barang secara cepat melalui perangkat genggam (tablet/PDA). Dengan pola daftar → detail, pekerja dapat mengetuk item inventori dan langsung melihat informasi lengkap (harga, stok, spesifikasi) tanpa harus membuka aplikasi berat berbasis web. Pengembangan lintas platform (Android/iOS) dengan satu kode JavaScript menghemat biaya dan waktu pengembangan hingga 50%.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Struktur Folder

Buat folder baru di dalam project bernama `screens`. Di dalamnya buat dua file:

1. `HomeScreen.js` (Daftar Inventori)
2. `DetailScreen.js` untuk detail barang



Gambar E.1 – struktur folder

E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

Buka folder `Home Screen.js` di VSCode.

```
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from 'react-native';

const HomeScreen = ({ navigation }) => {
  const inventory = [
    { id: '1', name: 'Laptop', price: 'Rp 10.000.000', stock: 5 },
    { id: '2', name: 'Mouse', price: 'Rp 150.000', stock: 20 },
    { id: '3', name: 'Keyboard', price: 'Rp 350.000', stock: 12 },
  ];

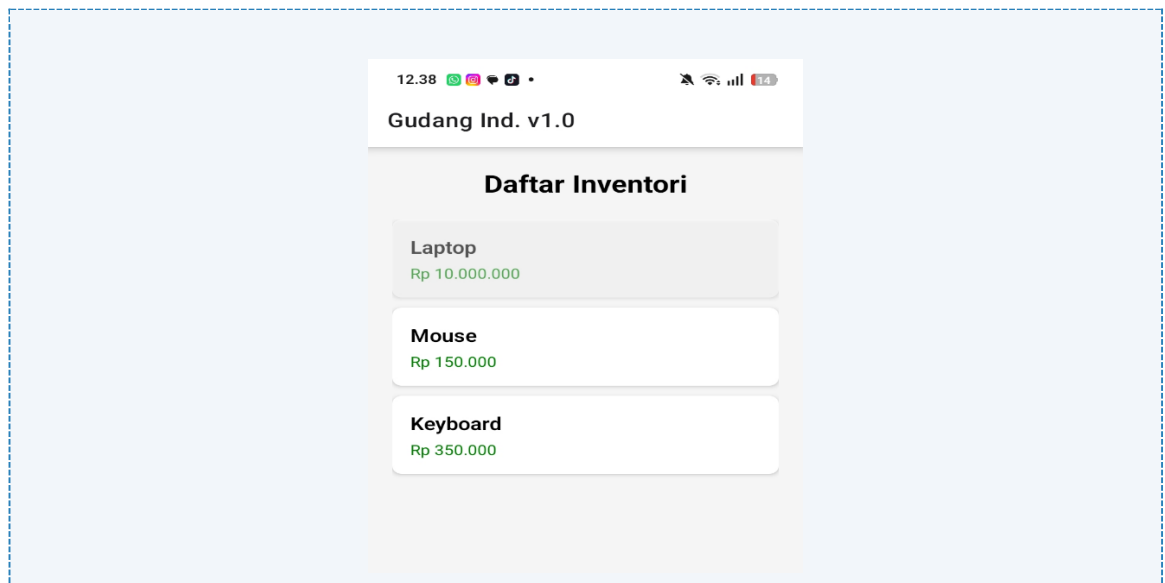
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.item}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemName}>{item.name}</Text>
      <Text style={styles.itemPrice}>{item.price}</Text>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Daftar Inventori</Text>
      <FlatList
        data={inventory}
        keyExtractor={(item) => item.id}
        renderItem={renderItem}
      />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
    textAlign: 'center',
  },
  item: {
```

```
        backgroundColor: 'white',
        padding: 15,
        marginBottom: 10,
        borderRadius: 8,
        elevation: 2,
      },
      itemName: {
        fontSize: 18,
        fontWeight: '600',
      },
      itemPrice: {
        fontSize: 14,
        color: 'green',
        marginTop: 5,
      },
    },
  ));

export default HomeScreen;
```



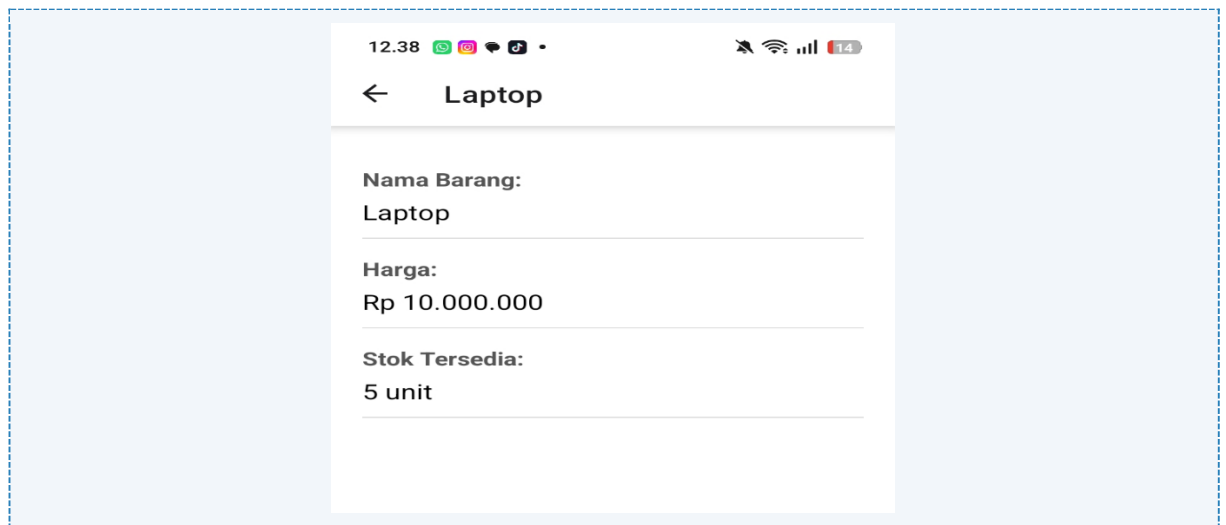
Gambar E.2 – HomeScreen.js

E.3 Langkah 3 – Membuat DetailScreen (Detail Barang)

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const DetailScreen = ({ route }) => {
  const { itemData } = route.params;
```

```
return (  
  <View style={styles.container}>  
    <Text style={styles.label}>Nama Barang:</Text>  
    <Text style={styles.value}>{itemData.name}</Text>  
  
    <Text style={styles.label}>Harga:</Text>  
    <Text style={styles.value}>{itemData.price}</Text>  
  
    <Text style={styles.label}>Stok Tersedia:</Text>  
    <Text style={styles.value}>{itemData.stock} unit</Text>  
  </View>  
>);  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    padding: 20,  
    backgroundColor: '#fff',  
  },  
  label: {  
    fontSize: 16,  
    fontWeight: 'bold',  
    marginTop: 15,  
    color: '#555',  
  },  
  value: {  
    fontSize: 18,  
    marginTop: 5,  
    paddingBottom: 10,  
    borderBottomWidth: 1,  
    borderBottomColor: '#ddd',  
  },  
});  
  
export default DetailScreen;
```

Screenshot Hasil:

Gambar E.3 – Detail Screen

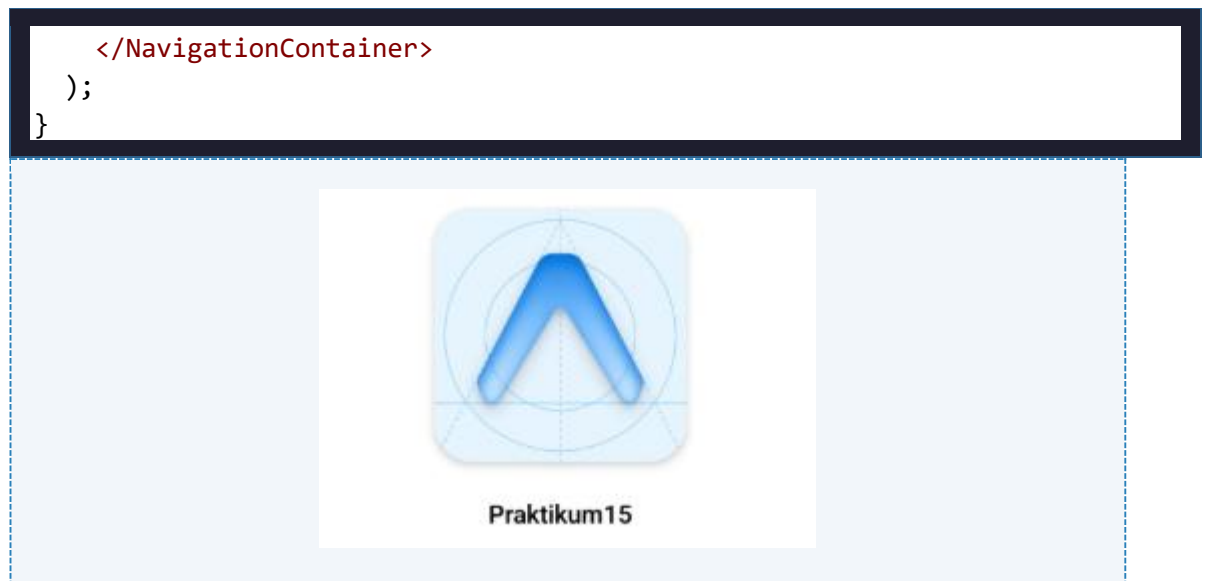
E.4 Langkah 4 – Mengonfigurasi Navigator di App.js

Buka App.js dan menambahkan seperti ini,

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Gudang Ind. v1.0' }}
        />
        <Stack.Screen
          name="Detail"
          component={DetailScreen}
          options={({ route }) => ({ title: route.params?.itemData.name ||
'Detail' })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

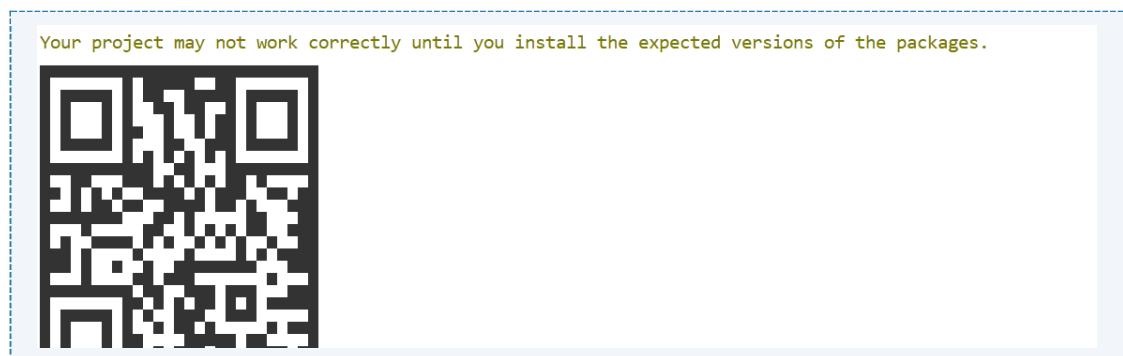


Gambar E.5 – npm intal library

E.5 Langkah 5 – Instalasi Library (Sebelum Menjalankan) dan Menjalankan Aplikasi

Jalankan perintah berikut di terminal project:

```
npm install @react-navigation/native@6 @react-navigation/native-stack@6
react-native-screens@3.22.0 react-native-safe-area-context@4.8.0 --legacy-
peer-deps
```



Gambar E.5 – npm intal library

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – (Conditional Rendering)

Pertanyaan / Instruksi Latihan:

`DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

saya mencoba merubah stok laptop menjadi 0, maka muncul request stok darurat

Jawaban / Kode Solusi:

```
import React from 'react';
import { View, Text, StyleSheet, Button, Alert } from 'react-native';

const DetailScreen = ({ route }) => {
  const { itemData } = route.params;

  const handleRequestStock = () => {
    Alert.alert('Permintaan Stok', `Permintaan stok darurat untuk
    ${itemData.name} telah dikirim ke gudang.`);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.label}>Nama Barang:</Text>
      <Text style={styles.value}>{itemData.name}</Text>

      <Text style={styles.label}>Harga:</Text>
      <Text style={styles.value}>{itemData.price}</Text>

      <Text style={styles.label}>Stok Tersedia:</Text>
      <Text style={[styles.value, itemData.stock === 0 &&
styles.stokHabis]}>
        {itemData.stock} unit
      </Text>

      {/* Conditional Rendering: Jika stok = 0, muncul tombol */}
      {itemData.stock === 0 && (
        <View style={styles.buttonContainer}>
          <Button
            title="Request Stok Darurat"
            color="orange"
            onPress={handleRequestStock}
          />
        </View>
      )}
    </View>
  );
};

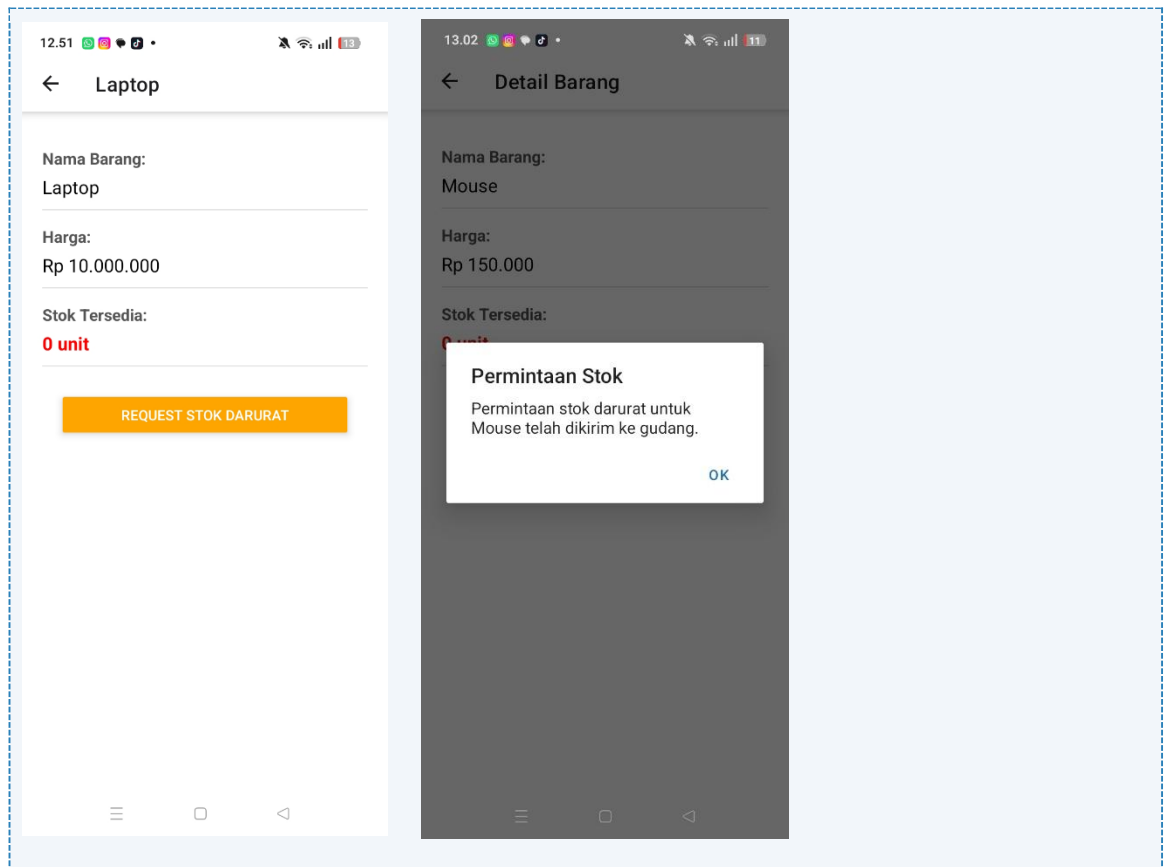
const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  label: { fontSize: 16, fontWeight: 'bold', marginTop: 15, color: '#555' },
  value: { fontSize: 18, marginTop: 5, paddingBottom: 10,
borderBottomWidth: 1, borderBottomColor: '#ddd' },
```

```

    stokHabis: { color: 'red', fontWeight: 'bold' },
    buttonContainer: { marginTop: 30, paddingHorizontal: 20 },
  });

export default DetailScreen;

```



Gambar F.1 – Request Stock Darurat

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Latihan 2 (Passing Data): Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi untuk file TambahScreen.js:

```

import React, { useState } from 'react';
import { View, Text, TextInput, StyleSheet, Button, Alert } from 'react-native';

const TambahScreen = ({ navigation }) => {

```

```
const [nama, setNama] = useState('');
const [harga, setHarga] = useState('');
const [stok, setStok] = useState('');

const handleSimpan = () => {
  if (!nama || !harga || !stok) {
    Alert.alert('Error', 'Semua field harus diisi!');
    return;
  }

  // Kirim data kembali ke HomeScreen
  navigation.navigate('Home', {
    barangBaru: {
      id: Date.now().toString(),
      name: nama,
      price: `Rp ${harga}`,
      stock: parseInt(stok),
    }
  });
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Tambah Barang Baru</Text>

    <Text style={styles.label}>Nama Barang</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: Scanner"
      value={nama}
      onChangeText={setNama}
    />

    <Text style={styles.label}>Harga (Rp)</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: 500000"
      keyboardType="numeric"
      value={harga}
      onChangeText={setHarga}
    />

    <Text style={styles.label}>Stok</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: 10"
    />
  </View>
);
```

```

        keyboardType="numeric"
        value={stok}
        onChangeText={setStok}
      />

      <View style={styles.buttonContainer}>
        <Button title="Simpan Barang" onPress={handleSimpan} />
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  title: { fontSize: 24, fontWeight: 'bold', textAlign: 'center',
marginBottom: 30 },
  label: { fontSize: 16, fontWeight: 'bold', marginTop: 15, color: '#333'
},
  input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8, padding:
12, marginTop: 5, fontSize: 16 },
  buttonContainer: { marginTop: 30 },
});

export default TambahScreen;

```

Jawaban / Kode Solusi untuk file homescreen agar menerima tambahscreen.js:

```

// Terima data dari TambahScreen
React.useEffect(() => {
  if (route.params?.barangBaru) {
    setInventory(prev => [...prev, route.params.barangBaru]);
    navigation.setParams({ barangBaru: null });
  }
}, [route.params?.barangBaru]);

const renderItem = ({ item }) => (
  <TouchableOpacity
    style={styles.item}
    onPress={() => navigation.navigate('Detail', { itemData: item })}
  >
    <Text style={styles.itemName}>{item.name}</Text>
    <Text style={styles.itemPrice}>{item.price}</Text>
    <Text style={styles.itemStock}>{item.stock} <Text style={styles.stokHabis}>

```

```

        Stok: {item.stock}
      </Text>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Daftar Inventori</Text>
      <FlatList
        data={inventory}
        keyExtractor={(item) => item.id}
        renderItem={renderItem}
      />

      { /* FAB (Floating Action Button) */ }
      <TouchableHighlight
        style={styles.fab}
        onPress={() => navigation.navigate('Tambah')}
      >
        <Text style={styles.fabText}>+</Text>
      </TouchableHighlight>
    </View>
  );
};

```

Update App.js

```

import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';
import TambahScreen from './screens/TambahScreen';

const Stack = createNativeStackNavigator();

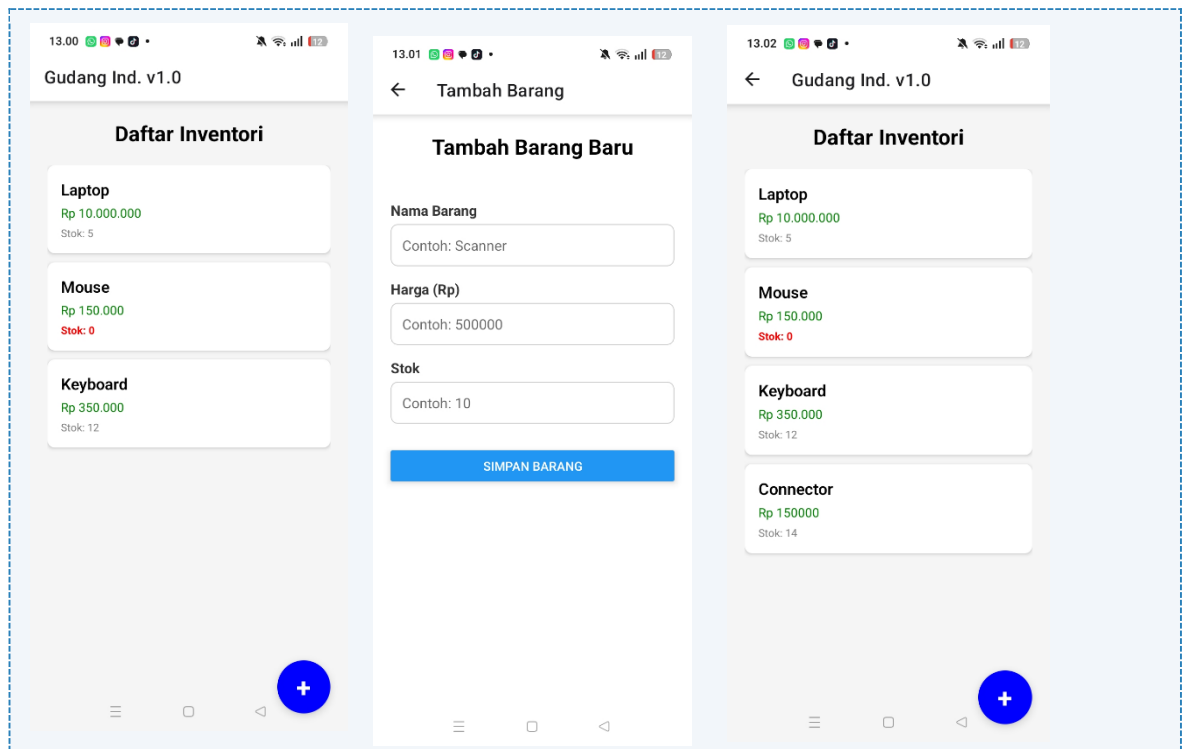
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} options={{ title:
'Gudang Ind. v1.0' }} />

```

```

        <Stack.Screen name="Detail" component={DetailScreen} options={{
title: 'Detail Barang' }} />
        <Stack.Screen name="Tambah" component={TambahScreen} options={{
title: 'Tambah Barang' }} />
    </Stack.Navigator>
</NavigationContainer>
);
}

```



Gambar F.2 – Tambah Screen

F.3 Aplikasi Inspeksi QC

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana. a. Home: Daftar item yang perlu diinspeksi. b. Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal). c. Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Layar	Fungsi
HomeScreen	Daftar item yang perlu diinspeksi
DetailScreen	Foto item, standar kualitas, dan dropdown status (Lolos/Gagal)
Logika	Jika pilih "Gagal", teks item di Home menjadi merah

Jawaban untuk item.js

```
export const items = [
  {
    id: '1',
    name: 'Panel Surya 100Wp',
    image: 'https://via.placeholder.com/300x200?text=Panel+Surya',
    standard: 'Efisiensi > 18%, Tidak ada retak, Tegangan output 18-22V',
    status: 'Belum Inspeksi'
  },
  {
    id: '2',
    name: 'Inverter 3000W',
    image: 'https://via.placeholder.com/300x200?text=Inverter',
    standard: 'Input DC 24V, Output AC 220V, THD < 3%',
    status: 'Belum Inspeksi'
  },
  {
    id: '3',
    name: 'Baterai Lithium 12V 100Ah',
    image: 'https://via.placeholder.com/300x200?text=Baterai',
    standard: 'Tegangan 12.6-13.2V, Tidak mengembung, Siklus > 2000',
    status: 'Belum Inspeksi'
  }
];
```

Jawaban untuk detailscreen

```
import React, { useState } from 'react';
import { View, Text, StyleSheet, Image, TouchableOpacity, Alert } from 'react-native';
import { Picker } from '@react-native-picker/picker';

const DetailScreen = ({ navigation, route }) => {
  const { itemData } = route.params;
  const [status, setStatus] = useState(itemData.status || 'Belum Inspeksi');

  const handleSimpan = () => {
    Alert.alert(
      'Konfirmasi',
      `Item "${itemData.name}" dinyatakan: ${status}`,
      [
        { text: 'Batal', style: 'cancel' },
        {
          text: 'Simpan',

```

```

        onPress: () => {
          navigation.navigate('Home', {
            updatedItem: { id: itemData.id, status: status }
          });
        }
      }
    ]
  );
};

return (
  <View style={styles.container}>
    </* Foto Item */>
    <Image source={{ uri: itemData.image }} style={styles.image} />

    </* Nama Item */>
    <Text style={styles.name}>{itemData.name}</Text>

    </* Standar Kualitas */>
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Standar Kualitas:</Text>
      <Text style={styles.standard}>{itemData.standard}</Text>
    </View>

    </* Dropdown Status */>
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Status Inspeksi:</Text>
      <View style={styles.pickerContainer}>
        <Picker
          selectedValue={status}
          onChange={(itemValue) => setStatus(itemValue)}
          style={styles.picker}
        >
          <Picker.Item label="⬤ Belum Inspeksi" value="Belum
Inspeksi" />
          <Picker.Item label="☑ Lolos" value="Lolos" />
          <Picker.Item label="✗ Gagal" value="Gagal" />
        </Picker>
      </View>
    </View>

    </* Tombol Simpan */>
    <TouchableOpacity style={styles.button} onPress={handleSimpan}>
      <Text style={styles.buttonText}>Simpan Hasil Inspeksi</Text>
    </TouchableOpacity>
  </View>
);

```

```

    );
  };

  const styles = StyleSheet.create({
    container: { flex: 1, padding: 20, backgroundColor: '#fff' },
    image: { width: '100%', height: 200, borderRadius: 10, marginBottom: 20, backgroundColor: '#eee' },
    name: { fontSize: 24, fontWeight: 'bold', textAlign: 'center', marginBottom: 20 },
    section: { marginBottom: 20 },
    sectionTitle: { fontSize: 16, fontWeight: 'bold', color: '#555', marginBottom: 8 },
    standard: { fontSize: 14, color: '#333', lineHeight: 20, backgroundColor: '#f9f9f9', padding: 12, borderRadius: 8 },
    pickerContainer: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8, overflow: 'hidden' },
    picker: { height: 50, width: '100%' },
    button: { backgroundColor: '#2196F3', padding: 15, borderRadius: 8, alignItems: 'center', marginTop: 20 },
    buttonText: { color: 'white', fontSize: 16, fontWeight: 'bold' },
  });

  export default DetailScreen;

```

Jawaban untuk Homescreen.js

```

import React, { useState } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from 'react-native';
import { items as initialItems } from '../data/items';

const HomeScreen = ({ navigation, route }) => {
  const [items, setItems] = useState(initialItems);

  // Update status item jika ada perubahan dari DetailScreen
  React.useEffect(() => {
    if (route.params?.updatedItem) {
      const updatedItems = items.map(item =>
        item.id === route.params.updatedItem.id
          ? { ...item, status: route.params.updatedItem.status }
          : item
      );
      setItems(updatedItems);
      navigation.setParams({ updatedItem: null });
    }
  }, [route.params?.updatedItem]);

```

```
const renderItem = ({ item }) => (  
  <TouchableOpacity  
    style={styles.item}  
    onPress={() => navigation.navigate('Detail', { itemData: item })}  
  >  
    <Text style={styles.itemName}>{item.name}</Text>  
    <Text style={styles.itemStandard} numberOfLines={1}>  
      Standar: {item.standard.substring(0, 50)}...  
    </Text>  
    <Text style={ [  
      styles.itemStatus,  
      item.status === 'Gagal' && styles.statusGagal,  
      item.status === 'Lolos' && styles.statusLolos  
    ]}>  
      Status: {item.status}  
    </Text>  
  </TouchableOpacity>  
);  
  
return (  
  <View style={styles.container}>  
    <Text style={styles.title}>Inspeksi QC - Gudang</Text>  
    <FlatList  
      data={items}  
      keyExtractor={(item) => item.id}  
      renderItem={renderItem}  
    />  
  </View>  
);  
};  
  
const styles = StyleSheet.create({  
  container: { flex: 1, padding: 20, backgroundColor: '#f5f5f5' },  
  title: { fontSize: 22, fontWeight: 'bold', marginBottom: 20,  
    textAlign: 'center', color: '#333' },  
  item: { backgroundColor: 'white', padding: 15, marginBottom: 10,  
    borderRadius: 8, elevation: 2 },  
  itemName: { fontSize: 18, fontWeight: 'bold', color: '#333' },  
  itemStandard: { fontSize: 12, color: '#666', marginTop: 5 },  
  itemStatus: { fontSize: 14, fontWeight: 'bold', marginTop: 8 },  
  statusLolos: { color: 'green' },  
  statusGagal: { color: 'red', fontWeight: 'bold' },  
});  
  
export default HomeScreen; import React, { useState } from 'react';
```

```
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from
'react-native';
import { items as initialItems } from '../data/items';

const HomeScreen = ({ navigation, route }) => {
  const [items, setItems] = useState(initialItems);

  // Update status item jika ada perubahan dari DetailScreen
  React.useEffect(() => {
    if (route.params?.updatedAt) {
      const updatedItems = items.map(item =>
        item.id === route.params.updatedItem.id
          ? { ...item, status: route.params.updatedItem.status }
          : item
      );
      setItems(updatedItems);
      navigation.setParams({ updatedAt: null });
    }
  }, [route.params?.updatedAt]);

  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.item}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemName}>{item.name}</Text>
      <Text style={styles.itemStandard} numberOfLines={1}>
        Standar: {item.standard.substring(0, 50)}...
      </Text>
      <Text style={[
        styles.itemStatus,
        item.status === 'Gagal' && styles.statusGagal,
        item.status === 'Lolos' && styles.statusLolos
      ]}>
        Status: {item.status}
      </Text>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Inspeksi QC - Gudang</Text>
      <FlatList
        data={items}
        keyExtractor={(item) => item.id}
        renderItem={renderItem}
      />
    </View>
  );
};
```

```

    />
  </View>
);
};

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#f5f5f5' },
  title: { fontSize: 22, fontWeight: 'bold', marginBottom: 20,
textAlign: 'center', color: '#333' },
  item: { backgroundColor: 'white', padding: 15, marginBottom: 10,
borderRadius: 8, elevation: 2 },
  itemName: { fontSize: 18, fontWeight: 'bold', color: '#333' },
  itemStandard: { fontSize: 12, color: '#666', marginTop: 5 },
  itemStatus: { fontSize: 14, fontWeight: 'bold', marginTop: 8 },
  statusLolos: { color: 'green' },
  statusGagal: { color: 'red', fontWeight: 'bold' },
});

export default HomeScreen;

```

Jawaban untuk tambahScreen.js

```

import React, { useState } from 'react';
import { View, Text, TextInput, StyleSheet, Button, Alert } from
'react-native';

const TambahScreen = ({ navigation }) => {
  const [nama, setNama] = useState('');
  const [harga, setHarga] = useState('');
  const [stok, setStok] = useState('');

  const handleSimpan = () => {
    if (!nama || !harga || !stok) {
      Alert.alert('Error', 'Semua field harus diisi!');
      return;
    }

    // Kirim data kembali ke HomeScreen
    navigation.navigate('Home', {
      barangBaru: {
        id: Date.now().toString(),
        name: nama,
        price: `Rp ${harga}`,
        stock: parseInt(stok),
      }
    });
  });

```

```
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Tambah Barang Baru</Text>

    <Text style={styles.label}>Nama Barang</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: Scanner"
      value={nama}
      onChangeText={setNama}
    />

    <Text style={styles.label}>Harga (Rp)</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: 500000"
      keyboardType="numeric"
      value={harga}
      onChangeText={setHarga}
    />

    <Text style={styles.label}>Stok</Text>
    <TextInput
      style={styles.input}
      placeholder="Contoh: 10"
      keyboardType="numeric"
      value={stok}
      onChangeText={setStok}
    />

    <View style={styles.buttonContainer}>
      <Button title="Simpan Barang" onPress={handleSimpan} />
    </View>
  </View>
);
};

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  title: { fontSize: 24, fontWeight: 'bold', textAlign: 'center',
marginBottom: 30 },
  label: { fontSize: 16, fontWeight: 'bold', marginTop: 15, color:
'#333' },
```

```
    input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8,
padding: 12, marginTop: 5, fontSize: 16 },
    buttonContainer: { marginTop: 30 },
  });

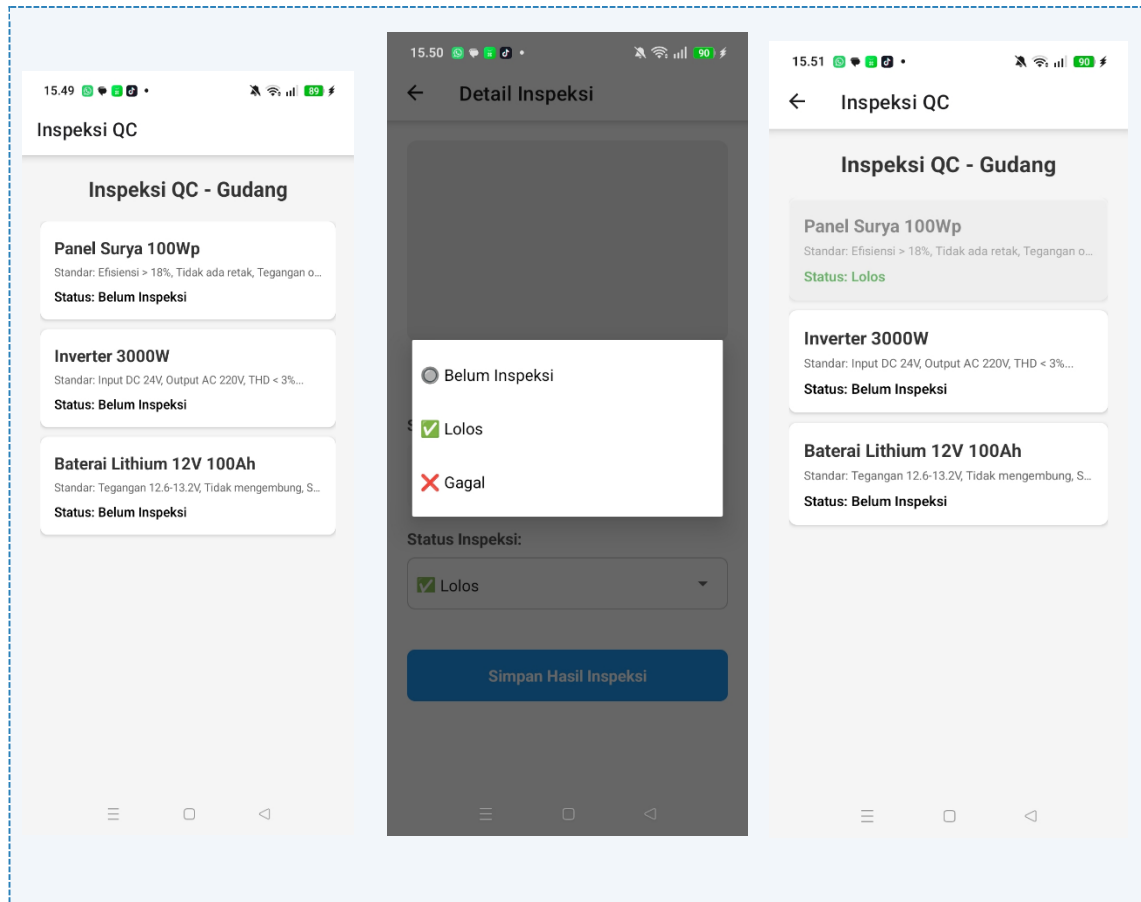
export default TambahScreen;
```

Jawaban untuk App.js:

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} options={{
title: 'Inspeksi QC' }} />
        <Stack.Screen name="Detail" component={DetailScreen} options={{
title: 'Detail Inspeksi' }} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```



Gambar F.3.– Tampilan Inventory QC

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan kesesuaian penuh dengan teori yang diberikan. Konsep **Stack Navigation** berhasil diimplementasikan di mana layar baru (DetailScreen) ditumpuk di atas layar lama (HomeScreen), dan tombol kembali secara otomatis melakukan *pop* pada history stack. Fungsi `navigation.navigate()` dan pengiriman data melalui params berjalan sesuai spesifikasi, di mana data item (nama, harga, stok) berhasil dikirim dari HomeScreen ke DetailScreen. Komponen `FlatList` digunakan untuk menampilkan daftar inventori dengan efisien karena hanya me-render item yang tampak di layar, sesuai dengan teori performa React Native.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Error "Project is incompatible with this version of Expo Go"	Membuat project baru dengan SDK yang sesuai versi Expo Go di HP (SDK 54) menggunakan <code>--template blank@sdk-54</code>

Error dependency tree (ERESOLVE) saat install @react-navigation	Menggunakan flag --legacy-peer-deps untuk mengabaikan konflik versi peer dependency
Error "Unable to resolve module @react-navigation/native"	Menghapus node_modules dan package-lock.json, lalu install ulang dengan versi yang kompatibel
Error "TurboModuleRegistry.getEnforcing: PlatformConstants"	Konflik versi React Native; solusi terbaik menggunakan Snack Expo untuk menghindari instalasi lokal
File App.js sudah diubah tapi HP tidak berubah	Clear cache dengan npx expo start -c dan restart Metro bundler
Error "Cannot read file tsconfig.json"	Diabaikan karena menggunakan JavaScript murni, file tsconfig.json dapat dihapus
Error "Project name cannot contain spaces"	Menggunakan nama project tanpa spasi seperti Praktikum15 bukan Praktikum 15

G.3 Kaitan dengan Penerapan di Industri

Aplikasi inventori dengan navigasi stack relevan untuk industri manufaktur dan logistik. Operator gudang dapat: Monitoring Stok Real-time – Lihat daftar dan detail stok dengan sekali sentuh. Permintaan Stok Darurat – Tombol request muncul otomatis saat stok habis. Tambah Barang Baru – Input item inventori langsung dari rantai produksi. Efisiensi Lintas Platform – Satu kode untuk Android & iOS, hemat biaya 40-50%.

H. KESIMPULAN

No	Kesimpulan
1	Navigasi berbasis stack di React Native menggunakan library @react-navigation/native-stack berhasil diimplementasikan untuk membuat aplikasi multi-layar (HomeScreen → DetailScreen) dengan mekanisme tumpukan layar.
2	Pengiriman data antar layar melalui params pada fungsi navigate() berhasil dilakukan, memungkinkan data item inventori (nama, harga, stok) ditampilkan di halaman detail sesuai dengan item yang dipilih.
3	Conditional rendering berhasil diimplementasikan pada DetailScreen, di mana tombol "Request Stok Darurat" hanya muncul jika nilai stok barang sama dengan 0, meningkatkan fungsionalitas aplikasi untuk kebutuhan darurat.
4	Fitur Floating Action Button (FAB) dan form tambah barang berhasil dibuat, memungkinkan pengguna menambahkan item inventori baru secara dinamis ke dalam daftar tanpa perlu me-refresh aplikasi.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chambers, J. M. (2014). <i>Object-Oriented Programming , Functional Programming and R</i> . 29(2), 167–180. https://doi.org/10.1214/13-STS452
[3]	Gunawan, R., Suherman, Y., & Wibowo, S. S. (2022). <i>Rancang Bangun Sistem Informasi Verifikasi Dan Validasi Data Pengajuan Tender Berbasis Web Menggunakan Model Terstruktur dan UML Menurut Instruksi Presiden Nomor 3 Tahun mengembangkan</i> . 16(62), 10–19.
[4]	Mulawarman Munsyir, S.E., S.Si., M. K., Eko Prasetyo Widhi, M. K., Eko Wahyudi, M. T., Dr. Bagus Setya Rintyarna, M. K., & Vandika, A. Y. (2024). <i>Belajar Algoritma dan Pemrograman dengan Javascript</i> .
[5]	React Navigation. (2026). <i>React navigation documentation</i> . https://reactnavigation.org/docs/getting-started/ .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —